© 2026 Alexander Pisani (a.h.pisani.research@gmail.com), with Claude (Anthropic).
Licensed CC BY 4.0. Please cite both author and AI collaborator — see CITATION.cff.

The Arithmetic Database: Relational Algebra as Multiplicative Structure on Prime-Encoded Tuples

Alexander Pisani & Claude (Anthropic)

a.h.pisani.research@gmail.com

April 2026

Abstract

We present a restricted specialisation of the resistance isomorphism framework [4] in which database entities are encoded as squarefree integers whose prime factors represent (attribute, value) pairs and participation in named relations. Under this encoding, the core operations of relational algebra reduce to elementary arithmetic on the encoded integers:

1. **Intersection** (shared attributes) is the greatest common divisor.
2. **Union** (combined attributes) is the least common multiple.
3. **Subset testing** (does entity A have all attributes of B) is divisibility.
4. **Symmetric difference** (attributes in exactly one of two entities) is lcm/ gcd.
5. **Many-to-many relationships** require no junction tables; relationship primes are additional factors of the entities' integers, and joins are divisibility tests.
6. **Similarity metrics** are computed as ratios of arithmetic quantities: cardinality Jaccard as $\Omega(\text{gcd})/\Omega(\text{lcm})$ (the standard set-theoretic measure), and log-weighted Jaccard as $\log(\text{gcd})/\log(\text{lcm})$ via the logarithmic isomorphism of [1] (which up-weights large-prime channels).

The restriction is the squarefree condition: within this paper, each prime appears in an entity's encoding with exponent 0 or 1. This sacrifices the rich multi-valued structure of [4] in exchange for two benefits: (i) the database operations become strictly bitwise / arithmetic, requiring only presence-absence detection per channel rather than phase or amplitude resolution; (ii) the mathematical structure maps transparently onto the physical architecture of [4, 5], with each prime becoming one wave-carrier channel and relational queries becoming single-stage parallel gates.

We present an implementation as a working Python module and three classes of application: **description-logic reasoning over bounded ontologies** (medical coding, legal compliance, chemistry), **real-time semantic classification** (sensor fusion, embedded decision systems), and **Grover-accelerated minimum-cover search** (an NP-hard problem that benefits from the combination of the database representation with the amplitude amplification of [7]). The third application is presented as the most compelling near-term demonstration target because it combines both architectural contributions of the framework into a problem with no efficient classical solution.

Keywords: relational algebra, prime encoding, squarefree integers, description logic, ontology reasoning, set cover, Grover amplification, wave computation

MSC (2020): 11A51, 11N37, 68P15, 68Q12, 03B70

---

1. Introduction

1.1 The problem

Relational databases impose a separation between *data* (tuples) and *relationships* (foreign keys, junction tables, indices). The separation is necessary because conventional storage — arrays, hash tables, B-trees — has no intrinsic notion of relationship; relationships are extrinsic structures maintained by the database engine.

The framework developed across [1–7] of this series provides an alternative. Integers, under the logarithmic isomorphism of [1] and the spectral decomposition of [3], carry relational structure intrinsically: two integers that share a prime factor share a common spectral dimension, and arithmetic operations on integers are operations on the underlying relational structure.

This paper specialises the framework to the case of *categorical data* — data whose attributes take values from a bounded finite set — and shows that the entire apparatus of relational algebra becomes a consequence of integer arithmetic under a specific encoding. The resulting representation has two properties that conventional databases lack:

- **Relationships are intrinsic.** Every relationship between two entities is recoverable by an arithmetic operation on their encoded integers. No external index structure is required.
- **Queries parallelise at the bit level.** The relational operations decompose into per-prime operations that are independent; in a physical wave-computer implementation of [4], these are simultaneous across all carrier channels, making relational queries constant-time regardless of the number of attributes.

1.2 The squarefree restriction

The full framework of [4] encodes each attribute as a prime and each value as the exponent of that prime, permitting arbitrarily many distinct values per attribute within a single channel. This is compact but requires phase or amplitude resolution to distinguish exponents, which is expensive in noise terms.

This paper restricts attention to the *squarefree* case: each prime appears in each entity's encoding with exponent either 0 (absent) or 1 (present). Every entity is then a product of a subset of a fixed prime pool, and the encoding is isomorphic to a bitvector over that pool.

The restriction costs expressive range: a multi-valued attribute with k possible values now consumes k primes in the pool, not one. For a lab PCB tier [Paper 7] supporting ~60 reliably-distinguishable primes, this caps the usable schema at roughly 15 attributes $\times$ 4 values each, or 30 binary attributes plus 30 relation primes. This is well below the scale of commercial databases but sufficient for the application classes identified in Section 7.

1.3 Relation to prior work

Source	Contribution used here
[1] Logarithmic Isomorphism	The $\log\gcd$ / $\log\operatorname{lcm}$ similarity measure

Source	Contribution used here
[2] Natural Operating System	Möbius structure; divisibility as primitive operation
[3] Structured Self-Information	Spectral decomposition $\sigma_p(n)$
[4] Multi-State Wave Computation	Physical architecture; character-based gates
[5] Spectral-Temporal Encoding	Multi-carrier channel allocation
[7] Geometry of Knowing	Grover amplification; measurement-basis framework

The classical literature the paper intersects with includes: description logic and subsumption reasoning [Baader et al.], bitmap indices in relational databases [O'Neil 1987], Bloom filters [Bloom 1970], and the Gödel-numbering tradition in mathematical logic. The novelty here is not the idea of prime encoding of tuples — Gödel numbering is a classical device — but the observation that under the squarefree restriction *and* with a native wave-computer implementation, the encoding transforms relational algebra from a database engineering problem into a hardware-level arithmetic operation.

---

2. The Squarefree Restriction and the Schema

2.1 Definitions

Definition 2.1 (Schema). A *schema* $\mathcal{S}$ consists of: 1. A set $A_{\mathcal{S}}$ of *attribute names*. 2. For each attribute $a \in A_{\mathcal{S}}$, a finite set V_a of *value names*. 3. A set $R_{\mathcal{S}}$ of *relation names* (used for many-to-many participation). 4. A *prime assignment* π: a bijection from $\{(a, v) : a \in A_{\mathcal{S}}, v \in V_a\} \cup R_{\mathcal{S}}$ into an ordered subset of the primes.

The *prime pool* of $\mathcal{S}$ is the image of π. Its cardinality is

$$|\pi(\mathcal{S})| = |R_{\mathcal{S}}| + \sum_{a \in A_{\mathcal{S}}} |V_a|.$$

Definition 2.2 (Entity). An *entity* e in schema $\mathcal{S}$ is a tuple $(\text{attrs}_e, \text{rels}_e)$ where attrs_e assigns to each attribute $a \in A_{\mathcal{S}}$ either a single value from V_a or no value (absence), and $\text{rels}_e \subseteq R_{\mathcal{S}}$ is the set of relations in which e participates.

Definition 2.3 (Encoding). The *encoded integer* of an entity e is:

$$n(e) = \prod_{a:\text{attrs}_e(a)=v} \pi(a, v) \cdot \prod_{r \in \text{rels}_e} \pi(r).$$

By construction, $n(e)$ is squarefree, and distinct (attribute, value) and relation assignments produce distinct factorisations.

2.2 Elementary properties

Proposition 2.1 (Injectivity). The encoding $e \mapsto n(e)$ is injective.

Proof. Each prime in the pool corresponds to a unique schema element by the bijection π. The prime factorisation of $n(e)$ recovers the set of schema elements e contains, and hence e itself. □

Proposition 2.2 (Squarefree closure). The set of encoded integers is closed under gcd and lcm, and both operations produce squarefree integers corresponding to valid (possibly partial) entities.

Proof. gcd and lcm preserve squarefreeness because they take minima and maxima of exponents respectively, and the inputs have exponents in $\{0, 1\}$. □

---

3. The Arithmetic-Relational Correspondence

We now state the central result: every standard relational-algebra operation on encoded entities reduces to an arithmetic operation on their integers.

3.1 The correspondence table

Let $n_A = n(e_A)$ and $n_B = n(e_B)$ be the encodings of two entities e_A, e_B in a common schema. Let $p_{av} = \pi(a, v)$ and $p_r = \pi(r)$ denote the primes for a (attr, value) pair and a relation.

Relational operation	Set-theoretic definition	Arithmetic realisation
"e has attribute a equal to v"	$\text{attrs}_e(a) = v$	$p_{av} \mid n(e)$
"e participates in relation r"	$r \in \text{rels}_e$	$p_r \mid n(e)$
Shared attributes	$S \cap T$	$\gcd(n_A, n_B)$
Combined attributes	$S \cup T$	$\text{lcm}(n_A, n_B)$
e_A is an "information subset" of e_B	$S \subseteq T$	$n_A \mid n_B$
Attributes unique to e_A	$S \setminus T$	$n_A / \gcd(n_A, n_B)$
Symmetric difference	$S \triangle T$	$\text{lcm}(n_A, n_B) / \gcd(n_A, n_B)$
Cardinality Jaccard similarity	$\lvert S \cap T \rvert / \lvert S \cup T \rvert$	$\Omega(\gcd(n_A, n_B))/\Omega(\text{lcm}(n_A, n_B))$
Log-weighted Jaccard similarity	(none — see remark)	$\log \gcd(n_A, n_B) / \log \text{lcm}(n_A, n_B)$
Hamming distance	$\lvert S \triangle T \rvert$	$\Omega(\text{lcm}/\gcd)$

where S, T denote the sets of (attribute, value, relation) elements of e_A and e_B, and $\Omega(n)$ denotes the number of prime factors of n counted with multiplicity — which for squarefree integers equals the count of distinct prime factors.

Remark on the two Jaccard variants. The table lists *two* similarity measures, both useful, both computed natively by arithmetic on the encoded integers, but ordering pairs differently. The cardinality Jaccard $\Omega(\gcd)/\Omega(\text{lcm})$ is the standard set-theoretic Jaccard coefficient and is the hardware-native measure when each prime channel contributes equally to similarity (one channel = one bit, regardless of which prime). The log-weighted Jaccard $\log \gcd / \log \text{lcm}$ inherits its weighting from the logarithmic isomorphism of [1] and assigns higher similarity to pairs that share *large* primes than to pairs that share an equal number of *small* primes. As a concrete example: two entities sharing primes $\{2, 3\}$ have cardinality Jaccard 0.33 and log-Jaccard ≈ 0.17; two entities sharing primes $\{67, 71\}$ over the same union size have cardinality Jaccard 0.33 but log-Jaccard ≈ 0.50. The

right choice depends on whether the schema treats all attributes as equally informative (cardinality) or treats large-prime channels as carrying more semantic weight (log-weighted).

Theorem 3.1 (Arithmetic-relational correspondence). *Let $\mathcal{S}$ be a schema and e_A, e_B entities in $\mathcal{S}$ with encoded integers n_A, n_B. Each of the relational operations in the table above is computed exactly by the corresponding arithmetic operation on n_A and n_B.*

Proof. For each entry, the claim follows from the fundamental theorem of arithmetic applied to squarefree integers: the set of prime factors of a squarefree integer is in bijection with the set of schema elements encoded. GCD takes the intersection of the prime-factor sets; LCM takes the union; divisibility expresses subset containment; etc. The cardinality Jaccard claim follows immediately because $\Omega(n)$ counts distinct prime factors (for squarefree n), so $\Omega(\text{gcd})$ and $\Omega(\text{lcm})$ are exactly $|S \cap T|$ and $|S \cup T|$. The log-weighted Jaccard claim follows from Paper 1: the logarithm sends multiplicative structure to additive structure, so $\log n$ counts prime factors weighted by their logs — and for a squarefree integer, $\log n = \sum_{p|n} \log p$, whose ratio on gcd /lcm is the log-weighted Jaccard coefficient. □

3.2 Many-to-many without junction tables

In classical relational databases, many-to-many relationships require an auxiliary "junction table" storing pairs (e, r) of entity participations. Queries involving the relationship join through this table.

Under the encoding of Definition 2.3, participation in a relation r is recorded directly by including $p_r = \pi(r)$ as a factor of $n(e)$. No junction table exists because no junction table is needed: the relationship is intrinsic to the entity's integer.

Corollary 3.1 (Relationship queries as divisibility). *Let $r_1, r_2, \ldots, r_k \in R_{\mathcal{S}}$. The set of entities participating in all of $r_1, \ldots, r_k$ is exactly*

$$\{e : \prod_i \pi(r_i) \mid n(e)\}.$$

The computational cost is one divisibility test per entity. In a physical wave implementation [4, 5], the divisibility test is a multi-channel presence-AND gate, which executes in constant time regardless of k.

---

4. Hardware Mapping

4.1 Primes as carrier channels

The physical interpretation follows directly from [4, 5]. Each prime p in the schema pool is assigned a distinct carrier frequency f_p. An entity's encoded integer corresponds to a multi-tone signal

$$s_e(t) = \sum_{p|n(e)} A_p \cos(2\pi f_p t + \phi_p)$$

where A_p and ϕ_p are amplitude and phase constants that can be set uniformly (the squarefree restriction means no per-prime exponent encoding is required — every prime present has the same intensity).

The database is then *literally* a bank of waveforms, one per stored entity. Queries are operations on these waveforms.

4.2 Relational operations as gates

- **Intersection (**gcd**)**: A per-channel AND gate. For each carrier frequency f_p, the output is the carrier if both input waveforms contain it, silenced otherwise. Physically implementable as correlator banks or multiplicative mixers followed by bandpass filters.
- **Union (**lcm**)**: A per-channel OR gate, or equivalently, linear summation followed by amplitude thresholding. Physically the simplest operation: analog summing junction.
- **Symmetric difference (XOR)**: Multiplicative mixing of the two waveforms with phase inversion on one, then filtering. Produces carriers only at channels where exactly one input has energy.
- **Divisibility (**$n_A \mid n_B$**)**: Pass s_A through s_B as a filter bank; if the output matches s_A, the divisibility holds. Physically a comparison of the AND output against s_A.

All four operations execute in parallel across all channels simultaneously. The operation time is bounded by the settling time of the analog circuitry, not by the number of attributes or the number of entities being compared pairwise.

4.3 Scaling within hardware tiers

From Paper 7 [Section 9], the maximum number of reliably-distinguishable carrier channels at each tier is:

Tier	q (channels)	Max attributes × 4 values	Max binary + relation primes
Breadboard (AD9833/AD8302)	~25	~6 × 4	~25
Lab PCB	~60	~15 × 4	~60
High-performance AC/RF	~200	~50 × 4	~200
Optical (multi-wavelength)	~4000	~1000 × 4	~4000

Scaling to millions of primes — the scale at which this architecture could plausibly compete with software knowledge graphs — would require breakthroughs in coherent wavelength density that are beyond the scope of this paper. The applications identified in Section 7 are therefore selected from domains where bounded schema sizes are not a limitation but a natural feature.

---

5. Worked Example: A Fruit-and-Region Database

To make the construction concrete, we work through a complete example corresponding to the Python implementation in the companion module `squarefree_db.py`.

5.1 Schema

Attributes: - `shape` {round, oblong, irregular} - `colour` {red, yellow, green, orange, brown} - `taste` {sweet, sour, bitter, savoury} - `category` {fruit, vegetable}

Relations: - `grown_in_europe`, `grown_in_asia`, `grown_in_americas`

The prime pool is assigned in order: primes 2 through 59 are used, covering the 14 (attribute, value) pairs plus 3 relations for a total of 17 primes.

5.2 Entity encoding

- **apple**: round, red, sweet, fruit; grown in Europe, Asia, Americas. $n(\text{apple}) = 2 \cdot 7 \cdot 23 \cdot 41 \cdot 47 \cdot 53 \cdot 59 = 1{,}940{,}284{,}738$.
- **banana**: oblong, yellow, sweet, fruit; grown in Asia, Americas. $n(\text{banana}) = 3 \cdot 11 \cdot 23 \cdot 41 \cdot 53 \cdot 59 = 97{,}309{,}113$.

5.3 Sample queries

- "Fruits with colour red":
$$p_{\text{red}} \mid n(e) \implies [\text{apple}].$$
- "Fruits that are sweet AND in category fruit":
$$p_{\text{sweet}} \cdot p_{\text{fruit}} \mid n(e) \implies [\text{apple, banana}].$$
- "Entities grown in Europe AND Asia":
$$p_{\text{europe}} \cdot p_{\text{asia}} \mid n(e) \implies [\text{apple}].$$
- "Jaccard similarity of apple and banana":
$$\frac{\log \gcd(n_A, n_B)}{\log \operatorname{lcm}(n_A, n_B)} = 0.599.$$

Each of these is a single arithmetic test on integers. In the wave hardware implementation, each is a single parallel gate across 17 carrier channels.

---

6. Theoretical Properties

6.1 Expressiveness

Proposition 6.1 (Expressive equivalence to finite relational algebra over fixed schemas). *For any finite relational schema with categorical attributes and relations of fixed arity, the squarefree prime encoding together with the arithmetic operations of Section 3 computes the same class of queries as finite relational algebra.*

Proof sketch. Any query in finite relational algebra decomposes into selection (attribute-value match), projection (drop unused attributes — corresponding to taking the gcd with a projector integer), join (intersection through relation primes), and set operations (union, difference — realised as lcm, $n/\gcd$). Each of these is realised in the arithmetic framework by Theorem 3.1 and Corollary 3.1. □

6.2 Connection to Paper 7: measurement bases

Each attribute in the schema defines a *measurement basis* in the sense of Paper 7 Section 7. Querying the value of attribute a for entity e is a projection of $n(e)$ onto the subspace spanned by the primes $\{\pi(a, v) : v \in V_a\}$. Different attributes yield different projections of the same underlying state — the same phenomenon as the three encodings of Paper 7 Section 7.5.

This is more than analogy. The Spectral Uncertainty Principle of Paper 7 Theorem 4.1 bounds the joint information extractable from mutually-non-factorable bases; here, that bound is vacuous because attributes correspond to disjoint prime subsets and are therefore mutually compatible measurements. The richer uncertainty structure emerges when attributes share primes, which under the squarefree restriction requires the prime pool to be shared across attributes — a case we do not pursue here but which connects this paper to the wider framework.

6.3 Connection to Paper 2: the master equation

The Möbius inclusion-exclusion identity of Paper 2 applies directly to squarefree integers. For any collection of entities $\{e_1, \ldots, e_k\}$, the *exclusive* intersection — attributes shared by exactly these entities and no others in the database — is computable by a Möbius-weighted alternating sum of gcd operations. This permits efficient evaluation of queries of the form "attributes characteristic of this subset of entities" and connects the database to the information-spectrum framework of Paper 3.

---

7. Applications

We now argue that the restricted framework is compelling *despite* its scale limitations, because the application classes where it excels are classes where schema sizes are naturally bounded, query latency matters more than database size, and relational reasoning is the bottleneck operation.

7.1 Description-logic reasoning over bounded ontologies

Description logic (DL) is the formal basis of reasoning systems for biomedical ontologies (SNOMED CT, the Gene Ontology), legal compliance systems, and semantic web standards (OWL). The core reasoning task is *subsumption*: given two concept expressions A and B, determine whether $A \sqsubseteq B$ (A is a subtype of B). In the squarefree encoding, if concepts are encoded as integers built from the primes of their defining attributes, then subsumption is exactly *divisibility*:

$$A \sqsubseteq B \iff n(B) \mid n(A).$$

This has two immediate consequences: - **Constant-time subsumption testing.** A hardware divisibility check takes one gate cycle regardless of ontology depth. - **Classification as a parallel operation.** Assigning an entity to the most-specific class that subsumes it — the central operation of DL classifiers like ELK, HermiT, FaCT++ — reduces to finding the integer in the class library with the highest gcd to the query integer. In hardware this is a parallel correlator bank.

Real ontologies are large (SNOMED CT has ~350,000 concepts), but the *primitive* attribute vocabulary is much smaller: a few hundred primitive qualifiers, body systems, and anatomical structures generate the combinatorial space. A schema fitting within the high-performance AC/RF tier (~200 primes) could cover the primitive vocabulary of a non-trivial clinical subdomain.

7.2 Real-time semantic classification

Embedded real-time systems — robot perception, autonomous vehicle classification, medical alarm prioritisation — require low-latency feature matching against a library of known patterns, often under tight power budgets. The squarefree encoding is a natural fit:

- Each feature detector produces a binary signal (present/absent).
- The detected-feature set is an integer.
- Classification against a library is a parallel gcd-max across stored reference integers.

The value proposition here is not "faster than a CPU" — modern CPUs are very fast at bitmap matching — but "constant-time, low-power, deterministic". A wave-based semantic coprocessor guarantees response time regardless of library size up to the hardware tier's channel budget, which matters for safety-critical real-time applications where worst-case latency is a hard requirement.

7.3 Grover-accelerated minimum set cover

The most compelling near-term application combines the database layer with the Grover amplification of Paper 7 to attack a classical NP-hard problem.

The problem. Given a family of sets $\{S_1, S_2, \ldots, S_m\}$ and a target set T, find the smallest collection $\mathcal{C} \subseteq \{S_1, \ldots, S_m\}$ such that $\bigcup_{S \in \mathcal{C}} S \supseteq T$. This is **Set Cover**, one of Karp's original 21 NP-complete problems.

The encoding. Each S_i becomes a squarefree integer $n_i = \prod_{e \in S_i} p_e$, where primes are assigned to elements. The target is $n_T = \prod_{e \in T} p_e$. A collection $\mathcal{C}$ covers T if and only if $n_T \mid \mathrm{lcm}(\{n_i : S_i \in \mathcal{C}\})$.

The oracle. The wave database computes "does this subset of sets cover the target" in a single arithmetic operation. An oracle marking all cover-producing subsets of size $\leq k$ is implementable as a two-stage pipeline: compute the LCM of the selected subset (union stage), test divisibility against n_T (cover stage).

The amplification. Grover amplification applied to the 2^m possible subsets finds a cover of size $\leq k$ in $O(\sqrt{2^m / N_k})$ queries, where N_k is the number of valid covers of size $\leq k$. Binary search over k finds the minimum cover size in $O(\log m \cdot \sqrt{2^m})$ total queries.

The sparse-marked regime. The quadratic speedup applies cleanly only in the regime where marked subsets (those satisfying the oracle condition) are rare relative to the search space — empirically, marked fraction $N_k/2^m \ll 1$ (less than approximately 10%). When marked subsets are dense (e.g., when the oracle marks all covers regardless of size, which can be 30–50% of the search space in random instances), the algorithm saturates in 1–2 iterations and the asymptotic advantage compresses or disappears. The minimum-finding application addressed here naturally satisfies the sparse-marked condition: the binary-search outer loop applies the oracle only to subsets of size $\leq k$ at each step, and minimum-size covers are typically a small fraction of all valid covers. Numerical verification across $m \in \{4, 6, 8, 10\}$: the dense-oracle regime ($N_k/2^m \approx 0.28$) produced fitted exponent $\alpha = 0.09$ with $R^2 = 0.12$; the sparse-oracle regime ($N_k/2^m \approx 0.05$, marking only minimum-size covers) produced $\alpha = 0.38$ with $R^2 = 0.92$, with deviation from the theoretical 0.5 attributable to integer rounding at small N and the residual dense contribution at $m = 4$. Independent verification of the same construction (P5 patent test suite, larger sample population) yields $\alpha = 0.51$ with $R^2 = 0.999$ in a pure minimum-cover regime.

A consequence worth noting for hardware planning: because the optimal Grover iteration count

in the sparse-marked regime is small (typically $\pi/4 \cdot \sqrt{N/N_k}$ with N_k tiny gives 2–9 iterations even for N up to 10^3), the noise budget required to demonstrate the algorithm at a given N is correspondingly relaxed. This compresses tier differentiation for minimum-finding applications relative to direct Grover search at the same N — though we do not characterise the limit of this effect in this paper.

Why this is compelling. 1. The problem is NP-hard: no classical polynomial-time algorithm exists, so a quadratic speedup is genuinely useful rather than merely being a speedup over an already-efficient algorithm. 2. The problem is directly applicable: minimum set cover appears in explanation generation (minimum set of rules explaining an observation), feature selection, test case minimisation, circuit design, and many other domains. 3. The encoding combines both pillars of the framework: the arithmetic database handles the structural representation and the oracle logic; Grover amplification handles the exponential search. 4. The demonstration scale is accessible: a 12-prime instance (covering sets of 12 elements with $2^{12} = 4096$ candidate subsets) is within the lab PCB tier and produces a measurable $\sim 64\times$ speedup.

A worked instance: given 12 observations (set elements) and 8 candidate rules (sets), find the minimum subset of rules explaining all observations. Classical brute force examines $2^8 = 256$ rule subsets. Wave-Grover examines $\sim \sqrt{256} = 16$ with constant-time per-subset cover-testing in hardware.

We propose this as the target demonstration for a university lab pitch: the problem is well-known, the speedup is measurable and falls within the breadboard's coherence budget, and it exhibits the integrated capability of the combined database + amplification architecture rather than either piece alone.

---

8. Limitations

8.1 Schema size

The hardest constraint: the number of reliably-distinguishable primes at each hardware tier. For categorical schemas with k attributes each taking v values plus r relations, the required prime count is $kv + r$. A lab PCB tier bounds usable schemas below ~60 primes; even an optimistic high-performance tier reaches ~200. Commercial-database schemas routinely involve thousands of distinct attribute values and are out of reach of any realistic near-term wave hardware.

8.2 Update semantics

The framework describes a read-optimised database. Updating an entity's attributes requires re-generating its carrier-signal representation, which is a hardware operation rather than a software index update. For write-heavy workloads this is a limitation; for read-heavy workloads (reference databases, classifier libraries, rule engines) it is not.

8.3 Range and numerical queries

The encoding is well-suited to categorical data but poorly suited to numerical range queries ("find all entities with value between 3 and 7 on attribute x"). Numerical values require either explicit binning into categorical buckets (losing resolution) or a different encoding entirely. A hybrid architecture

with a classical front-end for numerical queries and a wave back-end for categorical reasoning is likely the right near-term design.

8.4 Squarefree expressiveness

The squarefree restriction commits to presence-absence attribute values. The general framework of [4] supports arbitrary-valued attributes at the cost of more demanding measurement hardware. There is a continuum between the two, and the right position on that continuum depends on the application. This paper advocates the squarefree end for the simplicity of its hardware story; the full-framework end remains the target for applications requiring finer value distinctions.

---

9. Discussion

9.1 Relationships as data, not metadata

The central conceptual claim of the paper is that the prime encoding collapses the separation between data and relational metadata. In a conventional database, the schema and indices are structures external to the stored values; they are maintained by the database engine and must be kept consistent as values change. In the arithmetic database, the relationships *are* the values — every relational statement about the data is deducible from the data itself by arithmetic, with no external structure required.

This is not unique to our construction — Gödel numbering and bitmap indices share aspects of this property — but the combination with a native hardware implementation is new. In the wave computer, the integer *is* the physical signal, and arithmetic operations on integers *are* operations on physical signals. There is no encoding step separating the mathematical representation from the physical one. This tight coupling is what makes the "intrinsic relationships" claim operationally meaningful rather than merely philosophical.

9.2 Relation to knowledge graphs and vector embeddings

Modern knowledge representation systems fall into two families: *symbolic* (knowledge graphs with explicit relations) and *distributed* (vector embeddings with continuous similarity). The squarefree prime encoding sits in an unusual position between these families.

- Like knowledge graphs, it has discrete, interpretable structure: every prime factor corresponds to a specific semantic element.
- Like vector embeddings, it supports continuous similarity measures: Jaccard similarity, weighted gcds, information-theoretic distances.
- Unlike either, the similarity computations are arithmetic rather than either lookup-based or inner-product based.

Whether this hybrid position is valuable in practice is an open question. We observe only that it exists and that it becomes a distinct architectural option if the wave hardware reaches practical scale.

9.3 Connection to the meaning-oracle framework of Paper 7

The arithmetic database is a restriction of the meaning-oracle framework of [7] to presence-absence encoding. The meaning oracle permits multiple measurement bases over the same underlying state;

in the database, each attribute is such a basis, and a query selects which basis to measure in. The Spectral Uncertainty Principle does not constrain the database case because attributes partition the prime pool into disjoint sets — every attribute's basis is measurement-compatible with every other. When the pool is re-used across attributes (an extension not pursued here), the uncertainty structure would return.

---

10. Open Problems

1. **Dynamic schema extension.** Adding a new attribute requires extending the prime pool. Can the hardware support on-the-fly channel allocation, or must the pool be provisioned in advance?
2. **Approximate membership.** Bloom filters trade space for false-positive rate; does a squarefree analog exist, where approximate gcd suffices and hardware precision is relaxed?
3. **Compression via structural redundancy.** Ontology taxonomies have hierarchical structure (is-a chains). Can this be exploited to encode k concepts with fewer than k primes by sharing upstream primes across related concepts?
4. **Integration with classical databases.** What is the right architectural split between a wave-based semantic coprocessor and a classical primary database? The likely answer is an accelerator-style interface analogous to GPU offloading, but the details matter.
5. **Set cover in practice.** The theoretical speedup of Section 7.3 is clear; what is the practical breakdown threshold at which classical SAT-based approaches outperform the hardware, given current tier limitations?
6. **Connection to the qualia observation** (noted, not developed). Colour qualia exhibit structural isomorphism to squarefree multiplication: primary qualia combine into composite qualia but do not admit "squared" states. This is not the subject of this paper but suggests that the squarefree framework may have resonances in the foundations of perceptual representation that merit separate treatment.

References

[1] A. Pisani, "A unified algebraic framework for number theory, Boolean logic, and circuit topology via the logarithmic isomorphism," 2026.

[2] A. Pisani, "The Natural Operating System: Boolean computation via Möbius inversion and prime-encoded databases," 2026.

[3] A. Pisani, "Structured self-information: A prime coordinate decomposition of Shannon's information measure," 2026.

[4] A. Pisani and Claude (Anthropic), "Multi-state wave computation via Dirichlet characters: A generalisation of the Möbius interference architecture," 2026.

[5] A. Pisani and Claude (Anthropic), "Spectral-temporal wave encoding: A multi-wavelength architecture for linearly scalable wave computation," 2026.

[6] A. Pisani and Claude (Anthropic), "The geometric information gap: How arithmetic operations lose spatial structure and why $\zeta(2)$ governs the loss," 2026.

[7] A. Pisani and Claude (Anthropic), "The geometry of knowing: Partial spectral information, uncertainty, and quantum-analog computation in the resistance isomorphism framework," 2026.

[Baader et al.] F. Baader, D. Calvanese, D. McGuinness, D. Nardi, P. Patel-Schneider, eds., *The Description Logic Handbook: Theory, Implementation, and Applications*, 2nd ed., Cambridge University Press, 2007.

[O'Neil 1987] P. O'Neil, "Model 204 architecture and performance," *High Performance Transaction Systems*, Springer LNCS 359, 1987.

[Bloom 1970] B. H. Bloom, "Space/time trade-offs in hash coding with allowable errors," *Communications of the ACM* 13(7): 422–426, 1970.

[Karp 1972] R. M. Karp, "Reducibility among combinatorial problems," *Complexity of Computer Computations*, 85–103, Plenum, 1972.

Appendix A: Python Reference Implementation

The companion Python module `squarefree_db.py` provides a working implementation of the schema, entity, and arithmetic-database classes described in this paper. All examples in Section 5 are executable from the module's self-test (`python squarefree_db.py`).

Companion visualisations in `viz_arithmetic_db.py` produce: - `db_bitmatrix.png`: entities × (attr, value) presence matrix. - `db_relational_ops.png`: apple-vs-banana relational operations as coloured bitwise overlays. - `db_hardware_mapping.png`: the carrier-frequency signal interpretation, showing the AND-gate output for the shared channels.

A separate module `set_cover_grover.py` (planned) will implement the Grover-accelerated minimum set cover demonstration of Section 7.3 and produce scaling-law plots comparable to those of Paper 7 Section 6.

Appendix B: Notation

Symbol	Meaning
$\mathcal{S}$	Schema (set of attributes, values, relations, prime assignment)
$A_{\mathcal{S}}$	Set of attribute names in $\mathcal{S}$
V_a	Set of values for attribute a
$R_{\mathcal{S}}$	Set of relation names in $\mathcal{S}$
π	Prime-assignment function
$n(e)$	Encoded integer of entity e
p_{av}	Prime assigned to (a, v) pair
p_r	Prime assigned to relation r
$\gcd, \mathrm{lcm}$	Greatest common divisor, least common multiple

Symbol	Meaning
$\Omega(n)$	Number of prime factors of n (with multiplicity)
f_p	Carrier frequency assigned to prime p
$s_e(t)$	Multi-tone signal encoding entity e